

Gestor de Glosas — Rede Hospital Casa

Documento de Arquitetura Completo para Reprodução

1. VISÃO GERAL

Objetivo: Sistema de gestão de glosas hospitalares (billing disputes) para a Rede Hospital Casa. Permite upload de relatórios CSV semanais, cálculo automático de prazos de recurso por operadora, dashboard com semáforo de urgência (Vencido/Semana/Futuro), acompanhamento de resolução semana a semana, e envio de alertas por email.

Stack:

- Next.js 14 (App Router)
- TypeScript
- Prisma ORM + PostgreSQL
- Tailwind CSS
- NextAuth.js (Credentials Provider, JWT)
- Lucide Icons
- bcryptjs

Paleta de Cores (Rede Hospital Casa):

- Laranja principal: #F07F00
 - Azul marinho: #263578
 - Azul escuro: #112888
 - Teal: #006C6D
 - Ciano: #02B8BF
 - Cinza claro (fundo): #F2F2F2
 - Cinza escuro: #444444
-

2. SCHEMA DO BANCO DE DADOS (Prisma)

```

generator client {
  provider = "prisma-client-js"
  binaryTargets = ["native", "linux-musl-arm64-openssl-3.0.x"]
}

datasource db {
  provider = "postgresql"
  url      = env("DATABASE_URL")
}

model User {
  id          String    @id @default(cuid())
  name        String
  email       String    @unique
  password    String
  phone       String    @default("")
  isAdmin     Boolean   @default(false)
  canUpload   Boolean   @default(false)
  canEditPrazos Boolean @default(false)
  recebeNotificacoes Boolean @default(false)
  aprovado    Boolean   @default(false) // Requer aprovação do admin
  createdAt   DateTime @default(now())
  updatedAt   DateTime @updatedAt
}

model PrazoOperadora {
  id          String    @id @default(cuid())
  nomeOperadora String   @unique
  prazoDias   Int?      // null = pendente (prazo não definido)
  createdAt   DateTime @default(now())
  updatedAt   DateTime @updatedAt
}

model ArquivoCarregado {
  id          String    @id @default(cuid())
  slotPeriodo String?    // ATUAL, W1, W2, W3 ou null (não atribuído)
  dataUpload  DateTime @default(now())
  nomeArquivo String
  cloudPath   String    @default("")
  casos       CasoGlosa[]
  createdAt   DateTime @default(now())
}

model CasoGlosa {
  id          String    @id @default(cuid())
  arquivoId   String
  arquivo     ArquivoCarregado @relation(fields: [arquivoId], references: [id], onDelete: Cascade)
  hospitalReg String    @default("")
  hospitalAlias String   @default("")
  clienteNome String    @default("")
  hospitalCnpj String    @default("")
  operadoraGrupo String   @default("")
  operadoraNome String    @default("")
  convenioTipoPagamento String @default("")
  dataRecebimento DateTime?
  dataGuiaPagamento DateTime?
  dataSubmissaoAnalise DateTime?
  valorFaturado   Float    @default(0)
  valorRecebido   Float    @default(0)
  valorGlosa      Float    @default(0)
  guiaPrazoEnvioRecurso DateTime?
}

```

```

glosaNaoSubmetida      Float      @default(0)
glosaNaoAnalisada      Float      @default(0)
valorAceito            Float      @default(0)
valorRecurso           Float      @default(0)
valorRecursoRecebido   Float      @default(0)
valorGlosaMantida       Float      @default(0)
glosaMantidaNaoSubmetida Float      @default(0)
glosaNaoAnalisadaMantida Float      @default(0)
valorAceitoMantido      Float      @default(0)
valorRecursoMantido     Float      @default(0)
compRecebimento        String     @default("")
mes                    Int         @default(0)
ano                    Int         @default(0)
ccDataAtualizacao      DateTime?
dataLimiteCalculada     DateTime?
diasRestantes          Int?
status                 String     @default("SEM_PRAZO") // VENCIDO, SEMANA,
FUTURO, SEM_PRAZO

@@index([arquivoId])
@@index([operadoraGrupo])
@@index([hospitalAlias])
@@index([status])

```

3. ESTRUTURA DE COLUNAS DO CSV (28 colunas, encoding latin1)

O CSV de entrada é separado por vírgula, encoding `latin1`, com os seguintes headers:

```

hospital_reg, hospital_alias, cliente_nome, hospital_cnpj,
operadora_grupo, operadora_nome, convenio_tipo_pagamento,
r_recebimento_data_recebimento, g_guiia_data_pagamento,
g_guiia_submissao_para_analise_data, valor_faturado, valor_recebido,
valor_glosa, guia_prazo_envio_recurso, glosa_inicial_nao_submetida,
glosa_nao_analisada, valor_aceito, valor_recurso,
valor_recurso_recebido, valor_glosa_mantida,
glosa_mantida_nao_submetida, glosa_nao_analisada_mantida,
valor_aceito_mantido, valor_recurso_mantido, comp_recebimento,
mes, ano, cc_data_atualizacao_mais_recente

```

4. LÓGICA DE NEGÓCIO CENTRAL

4.1. Conceito de “Glosa Inicial em Andamento”

É o indicador principal do sistema. Definido como:

```
Glosa Inicial em Andamento = glosaNaoSubmetida + glosaNaoAnalisada
```

(“Não Submetida” + “Não Analisada” = total pendente de ação)

4.2. Cálculo de Prazo e Data Limite

Cada caso tem:

- `dataRecebimento` — data de recebimento da conta
- `prazoDias` — dias de prazo da operadora (vem de `PrazoOperadora`)
- `dataLimiteCalculada` = `dataRecebimento` + `prazoDias` (em dias)

4.3. Status Semáforo (calcStatus)

```
function calcStatus(dataLimite: Date | null, today: Date): string {
  if (!dataLimite) return 'SEM_PRAZO';
  const diffDays = dias entre dataLimite e today;
  if (diffDays < 0) return 'VENCIDO'; // 🔴 Prazo expirado
  if (dataLimite <= próximo domingo) return 'SEMANA'; // 🟡 Vence esta semana
  return 'FUTURO'; // 🟢 Vence após esta semana
}
```

Próximo domingo: calculado como o domingo seguinte ao dia atual (se hoje é domingo, pula para o próximo).

4.4. Normalização de Operadora

```
function normalizeOperadora(name: string): string {
  return name.normalize('NFD')
    .replace(/[\u0300-\u036f]/g, '') // remove acentos
    .replace(/\s+/g, '') // remove espaços
    .toUpperCase();
}
```

Exemplo: “SUL AMERICA” e “SULAMERICA” ambos viram “SULAMERICA”.

4.5. Slots de Período (4 slots fixos)

Slot	Significado
ATUAL	Semana corrente (dados mais recentes)
W1	Semana passada
W2	Retrasada
W3	Antiga (mais de 2 semanas atrás)

Limite: **máximo 4 arquivos** no sistema. Cada slot aceita 1 arquivo.

4.6. Regra Crítica de Recálculo: Apenas Slot ATUAL

O recálculo de status (tanto via `/api/recalculate` quanto via `/api/prazos` ao salvar um prazo) deve afetar **SOMENTE** os casos do slot `ATUAL`. Os slots históricos (W1, W2, W3) preservam seus status calculados no momento do upload, funcionando como “fotografia” daquela semana.

Motivo: Se recalcularmos W1/W2/W3 com a data atual, casos que eram “FUTURO” na semana passada aparecem erroneamente como “SEMANA” ou “VENCIDO” hoje, distorcendo o painel de evolução.

5. AUTENTICAÇÃO E PERMISSÕES

5.1. Auth Config

- NextAuth.js com `CredentialsProvider` (email/senha)
- Strategy: `JWT`
- Senha hasheada com `bcryptjs` (rounds: 12)
- Página de login: `/login`

5.2. Campos de Permissão no Token JWT

O JWT carrega: `id`, `isAdmin`, `canUpload`, `canEditPrazos`

5.3. Aprovação de Usuário

- Novos registros ficam com `aprovado: false`
- O primeiro usuário (admin master) recebe `aprovado: true` automaticamente
- Login é bloqueado para usuários não aprovados (retorna 403)
- Admin deve aprovar manualmente na tela de Usuários

5.4. Permissões por Funcionalidade

Funcionalidade	Permissão Necessária
Dashboard (visualização)	Qualquer autenticado
Upload CSV	<code>isAdmin</code> OU <code>canUpload</code>
Gerenciar Arquivos/Slots	<code>isAdmin</code> OU <code>canUpload</code>
Recalcular Prazos	<code>isAdmin</code> OU <code>canUpload</code> OU <code>canEditPrazos</code>
Editar Prazos por Operadora	<code>isAdmin</code> OU <code>canEditPrazos</code>
Gerenciar Usuários	<code>isAdmin</code>
Enviar Alertas Email	<code>isAdmin</code>

5.5. Middleware

Protege rotas `/dashboard/*` e `/admin/*` — redireciona para `/login` se não autenticado.

6. API ROUTES

6.1. `POST /api/signup`

- Cria usuário com `aprovado: false` (exceto primeiro = admin)
- Valida nome, email, senha (8-64 chars)
- Retorna status de aprovação

6.2. `POST /api/auth/login`

- Verifica credenciais + campo `aprovado`

- Retorna 403 se não aprovado

6.3. GET|POST|PATCH|DELETE /api/usuarios

- CRUD de usuários (admin only)
- Pode atualizar: `isAdmin`, `canUpload`, `canEditPrazos`, `recebeNotificacoes`, `aprovado`

6.4. POST /api/upload-csv

- Recebe `FormData` com campo `file`
- Limite de 4 arquivos no sistema
- Parse CSV (latin1, vírgula)
- Auto-detecta operadoras novas → cria `PrazoOperadora` com `prazoDias: null` (pendente)
- Calcula `dataLimiteCalculada`, `status`, `diasRestantes` para cada caso
- Insere em batch de 200
- Retorna `operadorasPendentes` e `totalOperadoras`
- Arquivo criado com `slotPeriodo: null` (usuário atribui depois)

6.5. GET|PATCH|DELETE /api/arquivos

- GET: lista arquivos + count de casos
- PATCH: atribui `slotPeriodo` (garante unicidade de slot)
- DELETE: exclui arquivo + cascade nos casos

6.6. GET|POST|DELETE /api/prazos

- GET: lista todos + auto-sync (detecta operadoras dos CSVs sem prazo → cria como pendente)
- POST: cria/atualiza prazo → recalcula casos **SOMENTE no slot ATUAL**
- DELETE: remove prazo

6.7. POST /api/recalculate

- Recalcula prazos de **SOMENTE o slot ATUAL**
- Usa data atual + prazoMap para recalcular `dataLimiteCalculada`, `status`, `diasRestantes`

6.8. GET /api/dashboard

- Parâmetros: `operadoras`, `hospitais`, `slot` (default: ATUAL)
- Retorna:
- `kpis`: { VENCIDO: {valor, count}, SEMANA: {valor, count}, FUTURO: {valor, count} }
- `porUnidade`: array com {hospital, VENCIDO, SEMANA, FUTURO}
- `porOperadora`: array com {operadora, VENCIDO, SEMANA, FUTURO, prazo}
- `empty`: boolean
- Valor = soma de (glosaNaoSubmetida + glosaNaoAnalisada) por status

6.9. GET /api/dashboard/evolucao

- Parâmetros: `operadoras`, `hospitais`
- Para cada slot (ATUAL, W1, W2, W3), retorna:
- `slot`, `nomeArquivo`, `dataUpload`
- `VENCIDO`, `SEMANA`, `FUTURO` (soma de glosa em andamento por status)

6.10. GET /api/dashboard/acompanhamento ★ NOVO

- Cruza dados de W1 (status SEMANA) com ATUAL
- Match por: `clienteNome` + `operadoraGrupo` + `dataRecebimento` + `compRecebimento`

- Agrega por operadora+hospital
- Classifica cada grupo como: RESOLVIDO, PARCIAL ou PENDENTE
- Retorna:
 - `available` : boolean
 - `resumo` : {total, valorTotal, resolvidos, valorResolvido, parciais, valorParcial, pendentes, valorPendente}
 - `items` : array com {operadora, hospital, valorW1, valorAtual, diff, resultado, casosW1}

6.11. GET /api/filters

- Retorna listas distintas de `operadoraGrupo` e `hospitalAlias` do slot ATUAL

6.12. POST /api/send-alerts

- Admin only
- Busca casos do slot ATUAL que são VENCIDO ou vencem em até 5 dias
- Gera email HTML rico com tabela de casos urgentes
- Envia para todos os usuários com `recebeNotificacoes: true`
- Usa API Abacus AI `sendNotificationEmail`

7. PÁGINAS E COMPONENTES

7.1. Páginas

Rota	Arquivo	Descrição
/	<code>app/page.tsx</code>	Redirect: autenticado → <code>/dashboard</code> , senão → <code>/login</code>
<code>/login</code>	<code>app/login/page.tsx</code>	Formulário de login com logo
<code>/signup</code>	<code>app/signup/page.tsx</code>	Registro com validação de senha
<code>/dashboard</code>	<code>app/dashboard/page.tsx</code>	Dashboard principal
<code>/admin/upload</code>	<code>app/admin/upload/page.tsx</code>	Upload CSV + gestão de slots
<code>/admin/prazos</code>	<code>app/admin/prazos/page.tsx</code>	CRUD de prazos por operadora
<code>/admin/usuarios</code>	<code>app/admin/usuarios/page.tsx</code>	CRUD de usuários + aprovação
<code>/admin/notificacoes</code>	<code>app/admin/notificacoes/page.tsx</code>	Envio de alertas por email

7.2. Dashboard (/dashboard) — Detalhamento

A página principal exibe, nesta ordem:

1. **Barra de Filtros** — MultiSelect de Operadoras e Hospitais
2. **3 KPI Cards (Semáforo)**
 -  Vencido (bg vermelho claro)
 -  A Vencer Semana (bg amarelo claro)
 -  A Vencer Futuro (bg verde claro)
 - Cada card mostra valor em R\$ e quantidade de casos, com animação de contagem
3. **Painel de Acompanhamento Semanal** 
 - Header: gradiente azul marinho → azul escuro
 - 4 cards resumo: Total W-1, Resolvidos (verde), Parciais (amarelo), Pendentes (vermelho)
 - Tabela detalhada: Operadora | Hospital | Valor W-1 | Valor Atual | Diferença | Status (badges)
 - Mensagem verde se tudo resolvido
 - Só aparece se W1 e ATUAL existirem e houver casos SEMANA no W1
4. **Tabela “Por Unidade”** — Hospital x {Vencido, Semana, Futuro, Total}
5. **Tabela “Por Operadora”** — Operadora x {Vencido, Semana, Futuro, Total, Prazo}
6. **Painel de Evolução** — Slots (ATUAL→W3) x {Vencido, Semana, Futuro, Total}

Tabelas têm: ordenação por coluna, paginação (8 por página), linha de totais.

7.3. Upload (/admin/upload)

- Área de upload de CSV
- Lista de arquivos com picklist de slot (ATUAL/W1/W2/W3/Não atribuído)
- Botão “Recalcular Prazos”
- Alerta amarelo quando há operadoras sem prazo definido (com link para /admin/prazos)
- Mapa visual dos slots ocupados

7.4. Prazos (/admin/prazos)

- Alerta amarelo se há operadoras pendentes (prazoDias = null)
- Lista separada: pendentes primeiro (com badge “ Pendente” e estilo amber)
- Edição inline: nome da operadora + dias de prazo
- Ao salvar, recalcula automaticamente os casos do ATUAL e mostra feedback

7.5. Usuários (/admin/usuarios)

- Lista com badges de permissões
- Modal de criação/edição
- Toggle de cada permissão: Admin, Upload, Prazos, Notificações, Aprovado
- Proteção contra auto-exclusão

7.6. Notificações (/admin/notificacoes)

- Botão para enviar alertas
- Instruções de como funciona (3 passos)
- Feedback de envio (sucesso/erro)

7.7. Componentes Customizados

Componente	Arquivo	Descrição
Header	app/components/header.tsx	Navbar com logo, links por permissão, mobile menu
KpiCard	app/components/kpi-card.tsx	Card com valor animado + contagem
MultiSelect	app/components/multi-select.tsx	Dropdown multi-seleção com "Todos"

8. LIBS UTILITÁRIAS

Arquivo	Funções
lib/prisma.ts	Singleton PrismaClient
lib/auth-options.ts	Config NextAuth (Credentials, JWT callbacks com roles)
lib/csv-parser.ts	parseCsvBuffer(buffer) , parseFloat2(val) , parseDate(val)
lib/prazo-calculator.ts	normalizeOperadora , getNextSunday , calcStatus , calcDiasRestantes
lib/formatters.ts	formatBRL(value) , formatNumber(value)
lib/aws-config.ts	Config S3 (bucket, region)
lib/s3.ts	generatePresignedUploadUrl , getFileUrl , deleteFile

9. ESTILOS E UI

9.1. globals.css

```
@tailwind base;
@tailwind components;
@tailwind utilities;

[data-hydratation-error] { display: none !important; }

:root {
  --orange: #F07F00;
  --navy: #263578;
  --dark-blue: #112888;
  --teal: #006C6D;
  --cyan: #02B8BF;
  --gray-light: #F2F2F2;
  --gray-dark: #444444;
}

body { background: #F2F2F2; color: #263578; }
```

9.2. tailwind.config.ts

Cores estendidas:

```
colors: {
  orange: { 500: '#F07F00', 600: '#D97200' },
  navy: { 500: '#263578', 600: '#1E2A60', 700: '#112888' },
  teal: { 500: '#006C6D', 600: '#005A5B' },
  cyan: { 400: '#02B8BF' },
}
```

9.3. Padrões Visuais

- Cards: `bg-white rounded-xl shadow-md`
- Headers de seção: fundo `bg-[#263578]` com texto branco
- Botões primários: `bg-[#F07F00]` com hover `bg-[#d96e00]`
- Link ativo no header: `bg-[#F07F00] text-white`
- Links inativos: `text-white/70 hover:text-white`
- Container máximo: `max-w-[1600px]`
- Loading spinner: `border-[#F07F00] border-t-transparent rounded-full animate-spin`

9.4. Assets em `/public`

- `logo-rede-casa.png` — Logo branco (para header escuro)
- `logo-rede-casa-dark.png` — Logo escuro (para páginas claras: login/signup)
- `favicon.svg` — Favicon
- `og-image.png` — Open Graph image

10. VARIÁVEIS DE AMBIENTE

```
DATABASE_URL=postgresql://...  
NEXTAUTH_SECRET=<secret>  
AWS_PROFILE=hosted_storage  
AWS_REGION=us-west-2  
AWS_BUCKET_NAME=<bucket>  
AWS_FOLDER_PREFIX=<prefix>/  
ABACUSAI_API_KEY=<key>  
WEB_APP_ID=<app_id>  
NOTIF_ID_ALERTA_DE_PRAZOS_PRXIMOS_AO_VENCIMENTO=<notif_id>
```

11. SEED DATA

11.1. Usuário Admin Master

- Nome: Pedro Ambrosio
- Email: pedro.ambrosio@redehospitalcasa.com.br
- Senha: Casa@2026!
- Todas as permissões + aprovado

11.2. Prazos Iniciais (20 operadoras)

Operadora	Prazo (dias)
BRADESCO	60
GEAP	30
PORTO SEGURO	60
UNIMED FERJ	30
AMIL	90
CORREIOS	60
CASSI	30
SULAMERICA	60
REAL GRANDEZA	30
KLINI	30
ELETROS	30
OMINT	60
MEDISERVICE	60
PETROBRAS	30
GOLDEN CROSS	60
VALE	30
ASSIM	30
FAPES	30
VIVEST	30
CABERJ	30

12. MIDDLEWARE

```
import { withAuth } from 'next-auth/middleware';
export default withAuth({ pages: { signIn: '/login' } });
export const config = { matcher: ['/dashboard/:path*', '/admin/:path*'] };
```

13. FLUXO OPERACIONAL

13.1. Fluxo Semanal

1. Operador exporta CSV do sistema de faturamento
2. Admin faz upload em `/admin/upload`
3. Atribui o arquivo ao slot correto (ATUAL, W1, etc.)
4. Sistema auto-detecta operadoras novas → alerta para configurar prazos
5. Admin configura prazos em `/admin/prazos`
6. Dashboard atualiza automaticamente com semáforo
7. Admin envia alertas por email via `/admin/notificacoes`

13.2. Rotação de Slots

Cada semana:

- Arquivo de W2 vai para W3 (ou é excluído)
- Arquivo de W1 vai para W2
- Arquivo de ATUAL vai para W1
- Novo CSV vai para ATUAL

13.3. Acompanhamento Semanal (Automático)

O painel cruza automaticamente os casos que eram “SEMANA” no W1 com os dados do ATUAL para mostrar se foram resolvidos, parcialmente resolvidos, ou continuam pendentes.

14. REGRAS IMPORTANTES

1. **Recálculo somente ATUAL** — Nunca recalculamos status de W1/W2/W3 com data atual
 2. **Aprovação obrigatória** — Novos registros ficam pendentes até admin aprovar
 3. **Normalização de operadora** — Sempre normalizar antes de comparar nomes
 4. **4 arquivos máximo** — Não permite upload se já tem 4
 5. **Slot único** — Cada slot aceita apenas 1 arquivo
 6. **Operadoras pendentes** — Auto-criadas com `prazoDias: null`, exibidas com destaque
 7. **Encoding Latin1** — CSVs usam encoding latin1 (não UTF-8)
 8. **Formato de moeda** — Sempre `pt-BR` com `R$`
 9. **Referência de status histórico** — W1/W2/W3 usam a data de upload como referência para cálculo
-

15. DEPENDÊNCIAS PRINCIPAIS (package.json)

```
"dependencies": {  
  "next": "14.x",  
  "react": "^18",  
  "react-dom": "^18",  
  "next-auth": "^4",  
  "@prisma/client": "^6",  
  "bcryptjs": "^2",  
  "lucide-react": "latest",  
  "tailwindcss": "^3"  
}  
"devDependencies": {  
  "prisma": "^6",  
  "typescript": "^5",  
  "@types/bcryptjs": "latest"  
}
```

16. ESTRUTURA DE DIRETÓRIOS

```

nextjs_space/
├── app/
│   ├── api/
│   │   ├── arquivos/route.ts
│   │   ├── auth/
│   │   │   ├── [...nextauth]/route.ts
│   │   │   ├── login/route.ts
│   │   ├── dashboard/
│   │   │   ├── route.ts
│   │   │   ├── evolucao/route.ts
│   │   │   ├── acompanhamento/route.ts
│   │   ├── filters/route.ts
│   │   ├── prazos/route.ts
│   │   ├── recalculate/route.ts
│   │   ├── send-alerts/route.ts
│   │   ├── signup/route.ts
│   │   ├── upload-csv/route.ts
│   │   ├── usuarios/route.ts
│   │   └── admin/
│   │       ├── notificacoes/page.tsx
│   │       ├── prazos/page.tsx
│   │       ├── upload/page.tsx
│   │       └── usuarios/page.tsx
│   ├── components/
│   │   ├── header.tsx
│   │   ├── kpi-card.tsx
│   │   └── multi-select.tsx
│   ├── dashboard/page.tsx
│   ├── login/page.tsx
│   ├── signup/page.tsx
│   ├── page.tsx
│   ├── layout.tsx
│   ├── providers.tsx
│   └── globals.css
├── lib/
│   ├── prisma.ts
│   ├── auth-options.ts
│   ├── csv-parser.ts
│   ├── prazo-calculator.ts
│   ├── formatters.ts
│   ├── aws-config.ts
│   └── s3.ts
├── prisma/
│   └── schema.prisma
├── scripts/
│   └── seed.ts
├── public/
│   ├── favicon.svg
│   ├── logo-rede-casa.png
│   ├── logo-rede-casa-dark.png
│   └── og-image.png
├── middleware.ts
├── tailwind.config.ts
└── tsconfig.json

```